

AI-Powered Automatic Retail Checkout System

Comprehensive Technical & Architecture Report

Presented By:

Muzzammil Idrees

Varisha

Date: May 2026

Course: Deep Neural Networks

Table of Contents

1. Executive Summary
2. Data Pipeline & Engineering
 - 2.1 Dataset Collection (MVTec D2S)
 - 2.2 Data Labeling & Conversion
 - 2.3 Data Validation & Versioning
 - 2.4 Feature Engineering & Augmentation
3. Deep Learning Model Architecture
 - 3.1 Architectural Selection (YOLOv8-seg vs Mask R-CNN)
 - 3.2 Hyperparameters & Configuration
 - 3.3 Training Procedure & Transfer Learning
 - 3.4 Model Evaluation & Metrics
 - 3.5 Model Packaging & Inference Engine
4. Codebase & System Architecture
 - 4.1 Training Scripts & Cloud Execution
 - 4.2 FastAPI Inference Server
 - 4.3 Database ORM & Glue Code
 - 4.4 Feature Transformations
5. Frontend Mobile POS Application
 - 5.1 Flutter Cross-Platform Architecture
 - 5.2 State Management (Riverpod)
 - 5.3 UI Components & Mask Rendering
6. Infrastructure & Deployment
 - 6.1 Compute & Hardware Acceleration
 - 6.2 Docker Orchestration
 - 6.3 Networking & Security
 - 6.4 Storage Solutions
 - 6.5 Monitoring & Logging
7. Conclusion

1. Executive Summary

The AI-Powered Automatic Retail Checkout System is an end-to-end computer vision application designed to modernize the retail checkout experience. Traditional barcode scanning is time-consuming and labor-intensive. This project introduces a friction-less checkout process leveraging Deep Neural Networks - specifically instance segmentation models - to automatically identify, count, and price groceries placed on a checkout counter in real-time.

The system is divided into three primary tiers:

- The Deep Learning Engine: Utilizing YOLOv8-seg (and optionally Mask R-CNN) trained on the MVTec D2S dataset to generate pixel-accurate segmentation masks for 60 grocery categories.
- The Backend API: A robust, asynchronous FastAPI application connected to a SQLite database, managing inventory, processing inference requests, and persisting transaction logs.
- The Frontend POS: A reactive Flutter application providing the Point of Sale interface, rendering real-time segmentation overlays, managing user checkout history, and handling inventory tracking.

2. Data Pipeline & Engineering

A robust data pipeline is critical for training highly accurate segmentation models. Our pipeline handles ingestion, transformation, augmentation, and validation of complex image data.

2.1 Dataset Collection (MVTec D2S)

We utilized the MVTec Densely Segmented Supermarket (D2S) dataset. This dataset is specifically designed for instance-aware semantic segmentation in an industrial retail setting. It contains high-resolution images of 60 distinct grocery categories, ranging from fresh produce (apples, bananas, tomatoes) to packaged goods (cereal bars, pasta, beverages).

The dataset represents challenging real-world scenarios including:

- Heavy occlusion where items are stacked on top of one another.
- Varying lighting conditions and reflections on plastic packaging.
- Deformable items (e.g., net bags of oranges) and visually similar items (e.g., different brands of water bottles).

2.2 Data Labeling & Conversion

The original MVTec D2S dataset provides annotations in the standard COCO JSON format. However, our primary model, YOLOv8-seg, requires annotations in a specific text-based format where polygon coordinates are normalized between 0.0 and 1.0.

We developed a custom data preparation script (`data_preparation.py`) to automate this conversion. The script performs the following operations:

- Parses `instances_train.json` and `instances_val.json`.
- Extracts polygon coordinates, filtering out complex Run-Length Encoded (RLE) masks.
- Normalizes absolute pixel coordinates by dividing by the image width and height.
- Maps the 60 COCO category IDs to a sequential 0-59 index required by Ultralytics.
- Generates individual `.txt` label files for every image in the dataset.

2.3 Data Validation & Versioning

To prevent data leakage and ensure unbiased evaluation, the dataset is strictly partitioned into training and validation splits. The training set features single-item images with clean backgrounds, while the validation set consists of multi-item scenes with occlusions to test generalization.

Data versioning and configuration are managed through a YAML manifest (`d2s.yaml`). This file acts as the single source of truth for the dataset, defining the root paths, number of classes (`nc: 60`), and the complete class mapping dictionary.

2.4 Feature Engineering & Augmentation

To prevent overfitting and improve the model's ability to generalize to unseen checkout counter configurations, we implemented an aggressive data augmentation pipeline during the training phase. These augmentations are applied dynamically in memory:

- **HSV Color Jittering:** Hue (0.015), Saturation (0.7), and Value/Brightness (0.4) adjustments simulate different camera sensors and lighting environments.
- **Geometric Transformations:** Random scaling (0.5x to 1.5x) and translations (10%) simulate varying camera distances and placements.
- **Rotations:** +/- 15 degree random rotations account for items being placed at arbitrary angles on the counter.
- **Flips:** 50% probability for both horizontal and vertical flips.
- **Mosaic Augmentation:** A 100% probability 4-image mosaic technique is employed. This combines four different training images into a single grid, forcing the model to detect items at smaller scales and simulating incredibly dense, cluttered checkout scenarios.

3. Deep Learning Model Architecture

3.1 Architectural Selection (YOLOv8-seg vs Mask R-CNN)

The project implements two distinct architectures to provide flexibility between inference speed and absolute accuracy. The primary architecture used in production is YOLOv8-seg.

Why Instance Segmentation? Standard object detection provides bounding boxes, which are insufficient for a retail checkout. If an apple is partially on top of a banana, their bounding boxes will overlap significantly, making it difficult to render accurate UI overlays or compute precise spatial relationships. Instance segmentation provides pixel-perfect polygon masks, separating overlapping objects distinctly.

YOLOv8-seg (Primary): Developed by Ultralytics, YOLOv8-seg is a state-of-the-art single-stage instance segmentation model. We utilize the 'nano' (YOLOv8n-seg) and 'medium' (YOLOv8m-seg) variants. It is optimized for real-time edge inference, making it ideal for a responsive POS system.

Mask R-CNN (Alternative): Implemented via Facebook's Detectron2 framework using a ResNet-50-FPN backbone. This is a two-stage detector (Region Proposal Network followed by a mask head). While slower, it serves as a robust baseline for comparative academic analysis.

3.2 Hyperparameters & Configuration

Hyperparameter tuning was conducted to optimize the YOLOv8-seg model for the D2S dataset. The final configuration is as follows:

- **Optimizer:** - Stochastic Gradient Descent (SGD)
- **Initial Learning Rate (lr0):** - 0.01
- **Final Learning Rate (lrf):** - 0.01 (using Cosine Annealing)
- **Momentum:** - 0.937
- **Weight Decay:** - 0.0005
- **Batch Size:** - 16
- **Input Image Size:** - 640x640 pixels
- **Epochs:** - 100
- **Early Stopping Patience:** - 20 epochs

3.3 Training Procedure & Transfer Learning

Training a segmentation model from scratch requires immense computational resources and data. To accelerate convergence, we utilized Transfer Learning. The model weights were initialized from a checkpoint pre-trained on the massive MS COCO dataset (80 object classes).

The training loop modifies the final classification head to output 60 classes instead of 80, and fine-tunes the mask generation branch. Training was executed using Google Colab's cloud infrastructure, specifically utilizing NVIDIA T4 and A100 Tensor Core GPUs. A unified train.py script orchestrates this, allowing seamless switching between YOLO and Detectron2 backends.

3.4 Model Evaluation & Metrics

The model is evaluated using the standard COCO metric suite, focusing on mean Average Precision (mAP).

- mAP@50: Measures precision where the Intersection over Union (IoU) between the predicted mask and ground truth is strictly > 50%.
- mAP@50-95: A stricter metric that averages mAP across IoU thresholds from 50% to 95% in 5% increments. This heavily penalizes sloppy mask boundaries.

During validation (validate.py), the system outputs per-class AP scores to identify problematic categories. Furthermore, Precision-Recall (PR) curves and Confusion Matrices are generated natively to visualize false positives (e.g., misclassifying 'Coca-Cola Light' as 'Coca-Cola Original').

3.5 Model Packaging & Inference Engine

In a production POS environment, latency is unacceptable. Model weights (.pt or .pth files) are statically packaged with the backend.

We engineered a Singleton InferenceEngine class in Python. During the FastAPI application 'lifespan' startup event, the model weights are loaded directly into RAM/VRAM. When an API request arrives, the engine is already warm, resulting in zero-latency initialization. Thread-safety is guaranteed using Python asyncio.Lock(), ensuring that concurrent HTTP requests do not cause race conditions within the PyTorch inference graph.

4. Codebase & System Architecture

The system is designed as a decoupled, microservice-style architecture. This separation of concerns ensures that the Deep Learning model, the API logic, and the UI can be scaled and updated independently.

4.1 Training Scripts & Cloud Execution

The repository contains a suite of CLI tools for offline model lifecycle management:

- `train.py`: The central entry point. Supports CLI arguments for model selection (`--model yolo/maskrcnn`), epoch count, and dataset paths.
- `validate.py`: Executes the COCO evaluator against the validation split and prints detailed tabular metrics.
- `predict.py`: A debugging utility that takes a single image, runs inference, draws bounding boxes/colored masks, and saves the output to disk.
- `train_colab.ipynb`: A Jupyter notebook wrapping the CLI tools, providing Google Drive mounting, dependency installation, and inline matplotlib visualization for execution on Google Colab.

4.2 FastAPI Inference Server

The backend API is built with FastAPI, chosen for its native async support, extreme performance (via Starlette/Pydantic), and automatic Swagger UI documentation.

Core Endpoints include:

- `POST /api/v1/checkout/scan`: The primary inference endpoint. It accepts a multipart/form-data image upload. It returns a complex JSON payload containing the detected items, calculated subtotals, tax rates, confidence scores, and raw polygon coordinates for rendering.
- `GET /api/v1/checkout/history`: Returns a paginated history of past transactions.
- `GET /api/v1/inventory/products`: Manages CRUD operations for the 60 grocery items, tracking stock quantities and prices.

4.3 Database ORM & Glue Code

We utilized SQLAlchemy 2.0 as the Object Relational Mapper (ORM) mapped to a SQLite database. This provides Pythonic, type-safe queries preventing SQL injection.

The schema consists of heavily normalized tables:

- `Products`: Stores name, category, unit price, and current `stock_quantity`.
- `Transactions`: Stores a UUID, timestamp, tax, and total amount.
- `TransactionItems`: Links products to a transaction with the quantity and frozen `unit_price_at_sale`.
- `InventoryLog`: An append-only audit trail logging every stock deduction (SALE) or addition (RESTOCK).

- AdminUser & OTPSession: Tables handling the secure MFA authentication flow.

4.4 Feature Transformations

Data must be transformed rapidly between the HTTP request, the model, and the HTTP response. Glue code in inference.py handles:

- Image Resizing: If a 4K image is uploaded, it is dynamically resized (maintaining aspect ratio) to a maximum dimension of 4096px before inference to prevent RAM exhaustion.
- Mask Extraction: The PyTorch binary mask tensors [N, H, W] are computationally heavy. We use OpenCV (cv2.findContours and cv2.approxPolyDP) to convert these dense binary masks into sparse lists of (x,y) polygon coordinates. This reduces the JSON payload size by 99%, allowing the Flutter frontend to receive the data instantly.

5. Frontend Mobile POS Application

The client-facing application is a cross-platform Point of Sale (POS) system built using the Flutter framework. It targets Android natively, but can also compile to Web and Windows Desktop.

5.1 Flutter Cross-Platform Architecture

Flutter was selected because it compiles to native ARM code, ensuring fluid 60fps animations which are critical when rendering complex UI overlays. The application handles camera feeds natively using the camera plugin, captures frames, compresses them to JPEG, and sends them to the FastAPI backend via the Dio HTTP client.

5.2 State Management (Riverpod)

Managing asynchronous state (API loading states, camera initialization, cart totals) is complex. We utilized Riverpod 2.x for robust, compile-safe state management.

- `checkoutProvider`: A `StateNotifier` that dictates the current phase of the checkout loop (Idle -> Capturing -> Uploading -> Analyzing -> Success/Error).
- `inventoryProvider`: A `FutureProvider` with auto-dispose caching that fetches the current product catalog and stock levels.
- `authProvider`: Manages the JWT tokens and MFA state for administrative access.

5.3 UI Components & Mask Rendering

The UI emphasizes a premium, modern aesthetic using Deep Blue (#1565C0) and Amber (#FF6F00) accents. The most technically complex component is the `SegmentationOverlay`.

When the JSON response arrives from the backend containing polygon arrays, Flutter's `CustomPainter` API is invoked. It loops through the polygons, applies a deterministic color palette based on the `class_id`, and draws semi-transparent filled paths directly on top of the original image. `RepaintBoundary` widgets are used to isolate this rendering layer, preventing the rest of the UI from dropping frames during the draw cycle.

The Receipt Panel utilizes staggered slide-in animations to present the detected items, unit prices, and grand totals clearly to the user/cashier.

6. Infrastructure & Deployment

The system is designed for modern cloud-native deployment patterns while remaining entirely functional on local edge hardware.

6.1 Compute & Hardware Acceleration

The backend utilizes PyTorch. If deployed on a machine with an NVIDIA GPU and CUDA drivers, PyTorch automatically shifts tensor operations to VRAM, reducing inference time from ~800ms (CPU) to ~80ms (GPU).

6.2 Docker Orchestration

To eliminate 'it works on my machine' issues, the backend is containerized. The Dockerfile utilizes python:3.11-slim as a minimal base image. It installs system-level dependencies required by OpenCV (libgl1, libglib2.0), copies the source code, creates volume mount points for the database and weights directories, and exposes port 8000. This ensures 100% reproducibility across environments.

6.3 Networking & Security

Networking is handled via RESTful HTTP. To support the Flutter Web client, Cross-Origin Resource Sharing (CORS) is explicitly configured via FastAPI middleware.

Security features include:

- Multi-Factor Authentication (MFA): Administrative screens are gated behind an email-based OTP system.
- Password Hashing: Administrator passwords are never stored in plaintext; they are hashed using SHA-256 with a unique random salt per user.
- Rate Limiting: The SlowAPI library implements an in-memory rate limiter (default 10 req/min/IP) to prevent Denial of Service (DoS) attacks on the computationally expensive inference endpoints.

6.4 Storage Solutions

Data storage relies on SQLite. To ensure high concurrency without database locking issues, Write-Ahead Logging (WAL) PRAGMA is explicitly enabled on the connection engine. This allows concurrent reads while a write (e.g., an inventory deduction) is occurring. Model weights are stored on the local file system (/weights) to prevent bloating the source control repository.

6.5 Monitoring & Logging

A custom HTTP middleware intercepts every incoming request. It tracks the start time, executes the request, and logs the HTTP method, path, status code, and latency in milliseconds. This is critical for monitoring the

performance of the AI model in production. Additionally, a `/api/v1/health` endpoint continuously reports the boolean status of the database connection and model readiness.

7. Conclusion

The AI-Powered Automatic Retail Checkout System demonstrates a successful integration of advanced deep learning techniques within a modern full-stack web and mobile application. By automating item identification via instance segmentation, the system dramatically reduces checkout friction.

The architecture's modularity ensures that the heavy AI inference engine is decoupled from the reactive POS interface, allowing both to scale independently. The inclusion of robust data augmentation pipelines guarantees high model accuracy even in cluttered retail environments. Future iterations could explore migrating to edge-TPUs (e.g., Google Coral) for localized, offline inference, entirely eliminating network latency from the checkout flow.